

Middleware: Types & Advanced Topics

Exam Study Notes – Multi-Tier Computing

Compiled from lecture slides (Topics not covered in core exam notes)

Peer-to-Peer Communications

Most early client/server applications were implemented using **low-level, conversational, peer-to-peer protocols** such as sockets, TLI, CPIC/APPC, NetBIOS, and Named Pipes. Programmers now use RPCs, MOMs, and ORBs which provide higher levels of abstraction. Conversational protocols should only be used by very demanding user applications or system-level software.

Key characteristics of peer-to-peer:

- Any computer can initiate a conversation with any other computer.
- The protocol is *symmetrical* — sometimes called “program-to-program.”
- The interface is “close to the wire” — it does not fully mask the underlying network from the programmer.
- The interface surfaces transmission timeouts, race conditions, and network errors, leaving it to the programmer to handle.

Sockets

The three most popular socket types are:

- **Stream sockets** — interface to the TCP protocol
- **Datagram sockets** — interface to the UDP protocol
- **Raw sockets** — interface to the IP protocol

A socket is identified by: `Net_Id . Host_Id . Port_ID` (mapped to an IP address).

NetBIOS and NetBEUI

NetBIOS is the premier protocol for LAN-based, program-to-program communications. Introduced by IBM and Sytek in 1984 for the IBM PC Network.

- Runs with almost no changes on every LAN.
- Used as an interface to a variety of stacks — including IBM/Microsoft LANs (NetBEUI), TCP/IP, XNS, Vines, OSI, and IPX/SPX.
- Support for NetBIOS exists on a wide variety of operating system environments.

NetBEUI is the protocol stack that comes with IBM and Microsoft LAN products (Windows for Workgroups, NT, LAN Manager, Windows 95/98, OS/2 Warp).

NetBEUI Strengths and Weaknesses:

Strengths: Offers powerful datagram and connection-oriented services; dynamic naming service based on discovery protocols.

Weaknesses:

- **Lack of a network layer** — main weakness.
- **Lack of security** — the broadcast mechanism used to dynamically “discover” names exposes names on unsecured networks and causes unwanted traffic.
- Most bridges and routers can filter discovery packets to block them from propagating to other networks.

MOM vs. RPC — Detailed Comparison

Comparing the RPC and messaging paradigms is like doing business via a *telephone call* (RPC) versus *exchanging letters or faxes* (MOM).

Feature	MOM: Messaging and Queuing	Remote Procedure Call (RPC)
Metaphor	Post office-like	Telephone-like
Client/Server time relationship	Asynchronous. Clients and servers may operate at different times and speeds.	Synchronous. Clients and servers must run concurrently. Servers must keep up with clients.
Client/Server sequencing	No fixed sequence.	Servers must first come up before clients can talk to them.
Style	Queued	Call-Return
Partner needs to be available	No	Yes
Load-balancing	Single queue can implement FIFO or priority-based policy.	Requires a separate TP Monitor.
Transactional support	Yes (some products). Message queue can participate in commit synchronization.	No. Requires a transactional RPC.
Message filtering	Yes	No
Performance	Slow. An intermediate hop is required.	Fast
Asynchronous processing	Yes. Queues and triggers are required.	Limited. Requires threads and tricky code for managing threads.

Additional MOM notes:

- Most MOM products make available a simple API set that runs on multiple operating system platforms.
- Most provide **persistent** (logged on disk) and **non-persistent** (in memory) message queues.
- Persistent messages are slower but can be recovered after power failures/system restarts.
- In both cases, messages can be either copied or removed from a queue.
- A message queue can be local to the machine or remote.
- System administrators can specify the number of messages a queue can hold and the maximum message size.

Enterprise Messaging System

An **enterprise messaging system** is a type of middleware that facilitates message passing between disparate systems or components in standard formats, often using **XML**, **SOAP**, or web services.

As part of an enterprise messaging system, **message broker software** may:

- Queue, duplicate, translate, and deliver messages to disparate systems or components.

Enterprise Service Bus (ESB)

Definition (Burton Group): ESB is “some type of integration middleware product that supports both message-oriented middleware and Web services.”

Functional Requirements of an ESB (layered bottom to top):

1. **Connectivity** — connecting disparate systems
2. **Routing** — directing messages to appropriate endpoints
3. **Transformation** — converting message formats
4. **Orchestration** — coordinating multi-step business processes
5. **Business Rules Engine** — applying business logic
6. **Business Monitoring** — tracking business events

Technical and business monitoring run alongside all layers.

ESB supports multiple application types:

- XTT Applications (via HTTP, JMS, JCA)
- J2EE Applications (via JMS, JCA)
- Packaged Applications (via JCA)
- .Net Applications (via WS-ReliableMessaging, C++, C#, VB)

Protocols at the lower level include: XTT, SOAP, WS-Reliability.

Abbreviations: EJB = Enterprise JavaBeans, JMS = Java Messaging Service, JCA = J2EE Connector Architecture

Additional Types of Middleware

Intelligent Middleware (IMW)

Intelligent Middleware provides real-time intelligence and event management through intelligent agents.

- Manages real-time processing of high-volume sensor signals and converts them into **intelligent and actionable business information**.
- Delivers actionable information via end-user power dashboards or pushes it to internal/external enterprise systems.
- Supports various heterogeneous types of hardware and software.
- Provides an API for interfacing with external systems.
- Uses a highly scalable, distributed architecture that embeds intelligence throughout the network to transform raw data into actionable knowledge.
- Can be packaged with tools to view and manage operations and build advanced network applications.

Content-Centric Middleware

Content-centric middleware offers a simple provider-consumer abstraction through which applications can issue requests for **uniquely identified content**, without worrying about where or how it is obtained.

- *Example: Juno* — allows applications to generate content requests associated with high-level delivery requirements; the middleware adapts the underlying delivery to access content from the best-suited sources.
- Similar to Publish/Subscribe middleware and the Content-Centric Networking (CCN) paradigm.

- *Example system:* **DDS** (Data Distribution Services)

Remote Procedure Call Middleware

RPC middleware enables a client to use services running on remote systems. The process can be **synchronous or asynchronous**.

Object Request Broker (ORB) Middleware

With ORB middleware, it is possible for applications to **send objects and request services** in an object-oriented system.

SQL-Oriented Data Access Middleware

SQL-oriented Data Access is middleware between **applications and database servers**.

Embedded Middleware

Embedded middleware provides communication services and software/firmware integration interface that operates between embedded applications, the embedded OS, and external applications.

Transaction Processing (TP) Monitors

TP Monitors provide tools and an environment to develop and deploy distributed applications. Programming controls most middleware facilities other than transactions.

Application Servers

Application servers are software installed on a computer to facilitate the serving (running) of other applications. Programming controls most middleware facilities.

SQL Integration and Interoperability

The Multi-Vendor Problem

In a multi-vendor SQL environment, the challenges are:

- Different SQL APIs
- Multiple database drivers
- Multiple FAPs (Format and Protocols) and no solid interoperability
- Multiple administration tools
- Worse: federated database joins, federated commits, or concurrent access to federated data

Vendor-Proprietary SQL Driver

A **SQL driver** is a thin client run-time element that:

- Accepts API calls, formats an SQL message, and handles exchanges with the server.
- The format of the SQL message and the handshake are called the **FAP (Format and Protocols)** — typically vendor-defined.
- Most vendors now support multiple protocol stacks. Some bundle stacks with their drivers; others support a common transport interface (like Sockets or Named Pipes).
- At the server side, the vendor typically provides “listeners” for the different stacks.

Gateways to Other Vendor Databases

- Most vendor-supplied gateways only provide an **intersection** of features supported by both databases — the *least common denominator*.
- Most vendor-supplied gateways only support **dynamic SQL**.
- They are only good for simple data extracts and queries — not for transaction processing.

Federated Database Example

Example: Customer data in Database A, Order data in Database B. A query joins Customer and Order tables across databases. Databases remain independent; the join is executed using federation middleware. This increases transparency but may add performance overhead.

Middleware Solution #1: The Common SQL Interface

- Provides a **single, common SQL API** shared by all client applications.
- Multiple vendor-specific drivers (Driver A, Driver B, Driver C) sit beneath the common API.
- Clients still use different vendor FAPs (FAP A, FAP B, FAP C) to communicate with the respective DBMS servers.
- Each vendor still maintains their own Database Administrator tools.

Middleware Solution #2: The Open SQL Gateway

- Provides a **single Gateway Driver** and **common SQL API** on the client side.
- Uses a single **Gateway FAP** on the network.
- **Gateway Servers** sit in front of each DBMS server (Vendor A, B, C) and translate the gateway FAP to each vendor's native protocol.

Middleware Solution #3: Federated SQL

Goal: Create a federated SQL environment that provides the same level of completeness as a single-vendor implementation.

- Uses a **Standard SQL API**, **Standard SQL Driver**, and a **Standard SQL FAP** across all vendors.
- Removes gateway catchers — improving server performance, reducing cost, and simplifying maintenance.
- Creates a **single database administration interface** (Single System Database Administrator).

Key requirement: To eliminate gateway catchers, the common FAP must either support a *superset* of all SQL dialects or *tolerate native SQL dialects* (pass-throughs). SQL vendors must replace their private FAPs with the common FAP.

Network Operating System (NOS) Middleware

Goal: To provide a **single system image** across large networks. NOSs are evolving from being a collection of independent workstations (able to communicate via a shared file system) to becoming real distributed computing environments that make the **network transparent to users**.

NOS Complexity scales with global reach:

Scope	Global Reach (People)
Department LAN	~500 (Low complexity)
Enterprise Network / Intranet	~100K
Extranet (VPN)	~500K
Internet	100+ Million (High complexity)

Desired Transparencies

1. **Location transparency** — You should not have to be aware of the location of a resource. Users should not have to include location information in the resource's name. (*Example violation: \\Machine\directory\file surfaces the server name.*)
2. **Namespace transparency** — Able to use the same naming conventions and namespace to locate any resource on the network. A universal namespace should be available.
3. **Logon transparency** — Provide a single password/authentication that works on all servers and for all services on the network.
4. **Replication transparency** — You should not be able to tell how many copies of a resource exist.
5. **Local/remote access transparency** — Able to work with any network resource as if it were on the local machine.
6. **Distributed time transparency** — No visible time differences across servers. The NOS must synchronize clocks on all servers.
7. **Failure transparency** — Shielded from network failures. The NOS must handle retries and session reconnects, and provide service redundancy for fault-tolerance.
8. **Administration transparency** — Only a single-system management interface is needed. The NOS must be integrated with local management services.

NOS: Extending the Local OS

- **Goal:** Practically anything that can be done on a local OS can be done remotely and transparently.
- The NOS allows applications written for the local OS to become networked without changing a line of code.
- Most NOSs allow clients running different OSs (DOS, Windows 98, Mac, Unix) to share files and other devices.
- **DCE Distributed File Service (DFS)** provides a single-image file system distributed across a group of file servers.
 - The DFS file naming scheme is **location-independent**.
 - Each file has a unique identifier consistent across the network.
 - File system uses the DCE global namespace like all other network resources.
 - Integrated with DCE security mechanisms.
 - Provides a **Local File System (LFS)** with advanced features including **replication facilities** that make the file system highly available.

Global Directory Services

The state of a client/server system is always in flux — users join and leave, services are added and moved, data is created and moved. Ideally, a distributed directory provides a **single image** usable by all network applications, including e-mail, system management, network inventory, file services, RPCs, distributed objects, databases, authentication, and security.

- A directory lets you search its objects **by attribute**.

- A **schema** describes the types of objects a directory may contain, as well as mandatory and optional attributes associated with each object type.
- Modern NOS directories have APIs and user interfaces that allow programs or humans to locate entities by querying on name or attributes.

Federated Directories — Creating a Unified Namespace

Federated directories: A system where **multiple independent directory services are linked together** so they appear as **one unified directory**, while each directory remains **autonomous and locally managed**.

Replica synchronization schemes:

- **Direct replication** — Any update to the master is immediately shadowed on all replicas.
- **Skulking** — Periodic propagation (e.g., once a day) of all master changes to all replicas.
- Typically, a directory maintains a **master copy** and **read-only shadow replicas**.
- **NetWare Directory Service (NDS)** — supports multi-master updates.
- **LDAP** (Lightweight Directory Access Protocol) — currently a single-master technology.

How to Interface to Directories

1. **Directory-specific APIs and class libraries** — Tightly coupled to a product. Example: Novell and Banyan provide APIs to their respective directories.
2. **LDAP and X.500 APIs** — Standards-based interfaces defined by the IETF, X/Open, and the CCITT.
3. **Java Classes (JNDI)** — The JavaSoft-defined **Java Naming and Directory Interface (JNDI)** provides a set of generic Java classes that let you interface to a variety of directory services such as LDAP, NDS, CORBA Naming, and NIS.
4. **Distributed object interfaces** — ORB-based interfaces that let you talk to directory servers across ORBs.
 - Allow you to pick your language, directory service, and platform.
 - *Example:* CORBA naming service (directory-neutral, language-neutral, platform-neutral).
 - In the COM universe: Microsoft's **Active Directory Service Interface (ADSI)** — directory-neutral and language-neutral for COM+ objects.
5. **Meta-directory services and scripts** — Glue products from companies like Zoomit, NetVision, and WorldTalk that synchronize data in existing directories via scripts.

X.500 and LDAP

- Historically, a directory service was tied to a single application (e.g., e-mail) or a specific OS (e.g., NT domains).
- A **universal directory** must be totally independent of both applications and NOSs.
- The original industry standard for universal directories was **CCITT's X.500** — but it proved too bulky and overly complicated.
- Today, the industry standard is **LDAP** — a downsized version of the original X.500, now the new “Esperanto” for directories.

End of supplementary middleware notes.